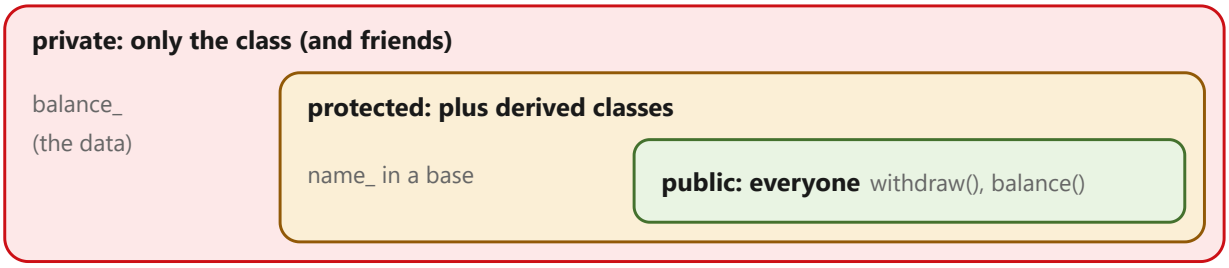
Object-Oriented Programming in C++

Classes, object lifetime, copying, inheritance and polymorphism: the parts that show whether you understand what a C++ object is. Assumes C++ Basics.

Classes and encapsulation



Hide the data, expose operations that keep the invariant true.

A **class** bundles data with the functions that use it. **Encapsulation**: keep data private, expose only operations that keep it valid.

```
class BankAccount {
public:                                    // the interface other code may use
    explicit BankAccount(double opening) : balance_(opening) {}

    bool withdraw(double amount) {
        if (amount ≤ 0 || amount > balance_) return false;   // invariant protected
        balance_ -= amount;
        return true;
    }
    double balance() const { return balance_; }

private:                                   // the implementation, hidden
    double balance_;
};

BankAccount acc{100.0};
acc.withdraw(30);          // fine
// acc.balance_ = 1e9;     // ERROR: private, nobody can break the invariant
```

SPECIFIER	WHO CAN ACCESS
`public`	everyone
`protected`	the class itself and classes derived from it
`private`	only the class itself (and its friends)

"Valid" is the class **invariant** (here: balance never negative). Hide the representation, let constructors build only valid objects, keep the public interface small. No invariant to protect? Use a plain `struct`.

> **STRUCT VS CLASS** Only the default access differs: `struct` public, `class` private. Use `struct` for plain data.

Member functions: `this`, `const`, `static`, `friend`

```
class Counter {
    int n_ = 0;
    static int instances_;                  // ONE copy shared by all Counter
objects
public:
    Counter() { ++instances_; }
    ~Counter() { --instances_; }

    Counter& add(int k) { n_ += k; return *this; }   // `this` is a pointer
to the current object
    int get() const { return n_; }                   // const: promises not
to modify the object
    static int instances() { return instances_; }    // no `this`: callable
as Counter::instances()

    friend std::ostream& operator<<(std::ostream&, const Counter&);   // may
touch private members
};
int Counter::instances_ = 0;                // a static data member must be
defined once
```

```
Counter c;
c.add(1).add(2);                          // chaining works because add
returns *this
```

- `const` **member function**: callable on const objects, cannot modify members (except `mutable`). Make getters const.
- **static member function**: belongs to the class, no `this`.
- `friend`: access to privates. Use sparingly.

Constructors and destructors

A **constructor** sets an object up; the **destructor** (`~Name()`) runs automatically when its lifetime ends. This is the basis of RAII.

```
class Widget {
    const int id_;
    std::string name_;
public:
    Widget() : Widget(0) {}                          // delegating
constructor
    Widget(int id, std::string n = "none")
        : id_(id), name_(std::move(n)) {}            // member initializer
list
    ~Widget() { /* release resources here */ }
};
```

- **Use the initializer list**: it initializes members directly. `const` and reference members can only be set there.
- **Members initialize in declaration order**, not list order (`-Wreorder`).
- `explicit` on a one-argument constructor blocks silent conversions.

Order of construction and destruction

Construction: base, members, derived body. Destruction: the exact reverse.

> KEY POINT Construct: base, members, derived body. Destruct: reverse.

Copying and the Rule of 3, 5 and 0

The compiler generates copy operations that copy every member. For a raw pointer that copies only the *address*: a **shallow copy**, and a bug.

Fix: a **deep copy**. **Rule of Three**: a custom destructor, copy constructor or copy assignment means you need all three.

```
class Buffer {
    int* data_;
public:
    Buffer(int v) : data_(new int(v)) {}
    ~Buffer() { delete data_; }                                  // 1. destructor
    Buffer(const Buffer& o) : data_(new int(*o.data_)) {}        // 2. copy constructor (deep)
    Buffer& operator=(const Buffer& o) {                         // 3. copy assignment (deep)
        if (this ≠ &o) { *data_ = *o.data_; }                    //    guard against a = a
        return *this;
    }
};
```

RULE	WHAT YOU WRITE	WHEN
Rule of 0	none of them	The goal. Hold resources in members that manage themselves: `std::vector`, `std::string`, `std::unique_ptr`.
Rule of 3	destructor, copy ctor, copy assign	The class owns a raw resource and must define copying.
Rule of 5	the 3 plus move ctor and move assign	Same, and you also want cheap moves (see Move Semantics).

```
class NoCopy {
public:
    NoCopy() = default;
    NoCopy(const NoCopy&) = delete;              // forbid copying outright
    NoCopy& operator=(const NoCopy&) = delete;
};
```

What the compiler still generates once you declare something

YOU DECLARE	COPYING	MOVING
nothing	generated	generated
a destructor, a copy constructor or a copy assignment	still generated (but deprecated)	**not generated**: an rvalue silently falls back to a copy
a move constructor or a move assignment	**disabled**	only what you declared

- A class can quietly lose its moves (a later logging destructor turns every move into a copy). Ask for defaults with `= default`; a polymorphic base needs them defaulted too.
- **Never `= delete` a move**: it kills the copy fallback too. For a move-only type, delete the *copy* operations.

> IN SHORT "Rule of Zero: `vector` and smart pointers make the generated members correct. Rule of Five only when wrapping a raw resource."

Inheritance

Public inheritance models **is-a**: a `Dog` works anywhere an `Animal` is expected.

```
class Animal {
public:
    explicit Animal(std::string name) : name_(std::move(name)) {}
    const std::string& name() const { return name_; }
protected:
    std::string name_;                    // visible to Dog, hidden from the
outside world
};

class Dog : public Animal {
public:
    explicit Dog(std::string name) : Animal(std::move(name)) {}   // call the
base constructor
    void fetch() const { std::cout << name_ << " fetches\n"; }
};
```

> **OBJECT SLICING** `Animal a = Dog("Rex");` chops off the Dog part and its virtual behaviour. Use pointers or references.

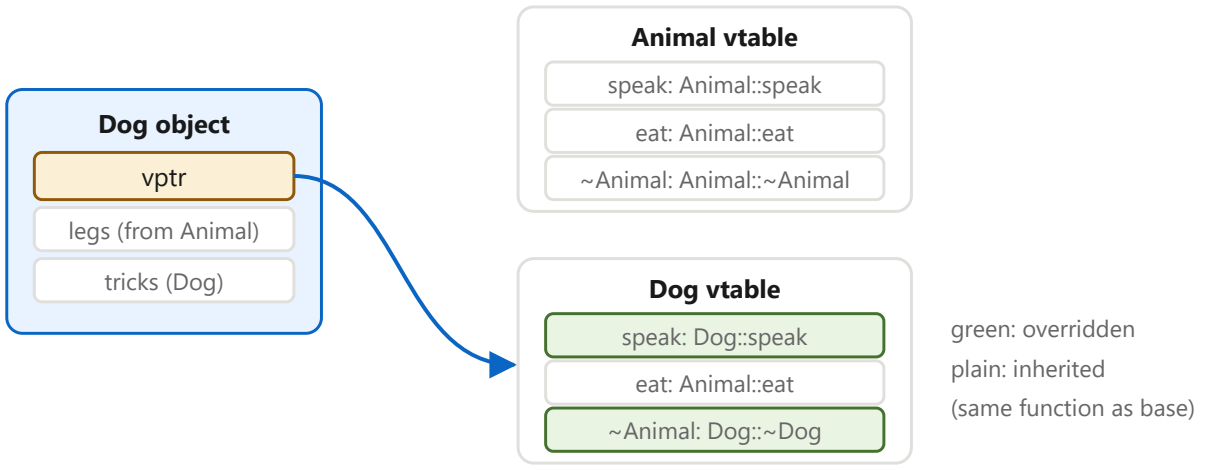
`final` forbids deriving. Prefer **composition** unless it is truly is-a.

Polymorphism and virtual functions

`p->speak()` through a base pointer reaches the *derived* behaviour only if `speak` is `virtual`. Without it the **static type** (the type of the pointer, known at compile time) decides; with it the **dynamic type** (the type of the object, known at run time) decides.

How it works: vtable and vptr

The compiler builds one **vtable** per class that has virtual functions: an array of function pointers, one **slot** per virtual function. A slot holds the *most derived override* of that function. A derived class's vtable starts as a copy of its base's, and every function the derived class overrides replaces the matching slot. Every object of such a class carries one hidden pointer, the **vptr**, normally at the start of the object, set to its class's vtable by the constructor.



One vtable per class, shared by all its objects; one vptr per object. Only the overridden slots differ.

- **Cost.** A class with virtual functions has a vptr in every object: `struct Plain { int legs; }` is 4 bytes, the same struct with one virtual function is 16 on a typical 64-bit platform (8 for the vptr, 4 for `legs`, 4 of padding). The data members come after the vptr.
- **The call.** `p->speak()` compiles to three steps: load the vptr from the object, index the vtable at the slot number that `speak` has in `Animal`, and jump to the address found there. The slot number is fixed at compile time because every derived vtable keeps the base's slot order. That is also why virtual calls usually cannot be inlined.
- **Only virtual functions have a slot.** Non-virtual functions are called directly, chosen by the static type.
- **A virtual destructor takes a slot too.** That is how `delete p` through an `Animal*` runs `Dog::~Dog`.
- **A pure virtual function** (`= 0`) still has a slot, but it holds no useful code; that is why the class cannot be instantiated.

Writing it: `override`, `final` and a virtual destructor

```
struct Shape {
    virtual double area() const = 0;
    virtual ~Shape() = default;            // ALWAYS virtual in a polymorphic
base
};
struct Circle : Shape {
    double r;
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }   // `override`:
the compiler checks you really override
};
struct Ball final : Circle {               // `final` on a class: nothing can
derive from it
    using Circle::Circle;
};
// `final` on a function stops further overriding:  double area() const
final;
```

TWO FAMOUS TRAPS

1. Always write `override`. If you mistype the signature (forget `const`), without it you silently create a *new* function instead of overriding.

2. Virtual calls inside a constructor or destructor do **not** reach the derived class. The constructor of `Base` sets the vptr to `Base`'s vtable, and only *after* it finishes does the constructor of `Derived` set it to `Derived`'s vtable. So while `Base`'s constructor runs, the object's dynamic type is `Base`, and `Base::init()` is called even if `Derived` overrides it. A destructor does the same in reverse.

Overloading, overriding and hiding

TERM	MEANING
Overloading	same name, different parameters, resolved at compile time
Overriding	a derived class replaces a *virtual* base function with the same signature, resolved at runtime
Hiding	a derived function with the same name (but not virtual / different signature) hides *every* base function of that name

```
struct B { void f(int); void g(); };
struct D : B { void f(double); };      // hides B::f(int), it does not
overload it

D d;
d.f(1);       // calls D::f(double): the int overload of B is not even
considered
d.g();        // fine: D has no g, so B::g is found
d.B::f(1);    // explicit qualification reaches the hidden one
// or add `using B::f;` inside D to bring B's overloads back into D's scope
```

Multiple base classes, `dynamic_cast` and the exact object layout are in The Object Model in Depth.

Abstract classes and interfaces

`= 0` makes a function **pure virtual**; a class with one is **abstract** (cannot be instantiated). Only pure virtuals and no data: an **interface**.

```
struct Logger {                                    // interface
    virtual void log(const std::string& msg) = 0;
    virtual ~Logger() = default;
};
struct ConsoleLogger : Logger {
    void log(const std::string& msg) override { std::cout << msg << '\n'; }
};

// Code depends on the abstraction, not the concrete class:
void run(Logger& out) { out.log("started"); }

// Polymorphic collection: unique_ptr owns the objects, no slicing, no manual
delete
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(2.0));
for (const auto& s : shapes) std::cout << s->area() << '\n';
```

`explicit`, conversion operators and copy-and-swap

```
struct Meters { double v; Meters(double x) : v(x) {} };              //
implicit: a double silently becomes Meters
struct Feet   { double v; explicit Feet(double x) : v(x) {} };       //
explicit: must be written out
takeMeters(3.0);                                                     //
compiles: surprising
// takeFeet(3.0);                                                    // ERROR
takeFeet(Feet(3.0));                                                 // OK

struct Handle {
    int fd;
    explicit operator bool() const { return fd ≥ 0; }                // a
conversion operator, but explicit
};
if (Handle h{3}) { }                                                 // OK:
an if is a "contextual" conversion to bool
// int n = h + 1;                                                    //
ERROR: no implicit conversion to bool or int
```

- **Mark single-argument constructors** `explicit` unless an implicit conversion is clearly wanted (compiled: `double` converts to `Meters` but not to `Feet`).
- `explicit operator bool` is the safe way to say "valid": it works in `if`, `while`, `&&` and `!`, but not in arithmetic or in comparisons with other types (the old "safe bool idiom" is obsolete). The standard streams and `std::optional` use it.
- A non-explicit `operator int()` lets your class take part in arithmetic and overload resolution in ways you did not plan.

Copy-and-swap

```
class Buffer {
    std::size_t n_; int* p_;
public:
    Buffer(const Buffer& o);                                    // deep
copy (written once)
    friend void swap(Buffer& a, Buffer& b) noexcept { std::swap(a.n_, b.n_);
std::swap(a.p_, b.p_); }
    Buffer& operator=(Buffer other) noexcept {                  // by
VALUE: the copy is made before we touch *this
        swap(*this, other);                                     // steal
the copy's resources
        return *this;                                           //
`other` dies and frees our OLD resources
    }
};
```

- **Strong exception guarantee:** if the copy throws, it happens before `*this` is touched.
- **Self-assignment safe** without an `if (this ≠ &o)` check (compiled: `b1 = b1` worked).
- One operator serves copy *and* move assignment (Move Semantics). Cost: a move-assignment written by hand can be a little faster.

Operator overloading

Overload operators only when the meaning is obvious (`+` on vectors, `==` on values, `<<` to print).

```
struct Vec2 {
    double x{}, y{};

    Vec2& operator+=(const Vec2& o) { x += o.x; y += o.y; return *this; }
// member: modifies *this
    bool operator==(const Vec2& o) const { return x == o.x && y == o.y; }
};
Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }   // non-member
built from +=

std::ostream& operator<<(std::ostream& os, const Vec2& v) {   // MUST be non-
member:
    return os << '(' << v.x << ", " << v.y << ')';             // the left
operand is the stream
}
```

- Non-mutating binary operators (`+`, `==`): **non-member**, so both sides convert. `+=`: member.
- Keep usual semantics: `==` with `≠`, `<` a strict weak ordering. C++20 `⟺` generates all comparisons.
- No new operators; `::` `.` `.*` `?:` cannot be overloaded.

Callable objects

```
struct Greater {                                          // a functor: an
object you can call like a function
    bool operator()(int a, int b) const { return a > b; }
};
std::sort(v.begin(), v.end(), Greater{});                 // a lambda is a
compiler-made functor
```

`operator()` makes a class callable (the idea behind lambdas, `std::function` and comparators). Conversion operators, and why to mark them `explicit`, are covered earlier, in *explicit, conversion operators and copy-and-swap*.

User-defined literals

```
constexpr long long operator""_km(unsigned long long v) { return
static_cast<long long>(v) * 1000; }   // kilometres to metres
constexpr double operator""_pct(long double v) { return static_cast<double>
(v) / 100.0; }              // percent to a fraction

auto dist = 4_km;         // 4000 (compiled)
auto rate = 25.0_pct;     // 0.25
using namespace std::literals;
auto name = "abc"s;       // std::string; also 5ms, 2s (chrono) and "view"sv
```

A literal suffix is an **operator** you define: `operator""_name`. User suffixes must start with an underscore (the others are reserved for the standard library). Use them for units (kilometres, percent, bytes) so the meaning is in the code, not in a comment.

Composition, relationships and design



Prefer composition; inheritance is the strongest coupling.

"Is it a, or does it have a?" Prefer **composition** unless you need polymorphism.

RELATIONSHIP	MEANING	C++ SHAPE	LIFETIME
Inheritance (is-a)	Dog is an Animal	`class Dog : public Animal`	one object
Composition (owns)	Car has an Engine, which dies with the car	member `Engine e;`	part dies with whole
Aggregation (uses, shared)	Team has Players who exist independently	`Player*` / `std::shared_ptr` member	independent
Association	Doctor treats Patients	reference or pointer parameter	independent

LISKOV SUBSTITUTION Code using a base must keep working with any derived type. If `Square : Rectangle` breaks on setting the width, inheritance was the wrong model: it must mean "behaves like", not "shares code".

GOING DEEPER Multiple and virtual inheritance, the diamond problem, RTTI and private inheritance are in The Object Model in Depth.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Classes and encapsulation	C++ Primer	7.1 Defining abstract data types	283 to 296
	C++ Primer	7.2 Access control and encapsulation	297 to 299
	C++ Primer	7.4 Class scope	311 to 316
	Programming: Principles and Practice	8.2, 8.3 Classes and members, interface and implementation	236 to 238
	Programming: Principles and Practice	8.7 Class interfaces	250 to 258
Member functions: this, const, static, friend	C++ Primer	7.1 Defining abstract data types	283 to 296
	C++ Primer	7.3 Additional class features	300 to 310
	C++ Primer	7.6 static class members	329 to 334
	Programming: Principles and Practice	15.6 to 15.8 Element access, an example list, the this pointer	464 to 473
	C++ Move Semantics	Chapter 5 Overloading on reference qualifiers	97 to 106
Constructors and destructors	C++ Primer	7.5 Constructors revisited	317 to 328
	C++ Primer	15.7 Constructors and copy control	651 to 658
	Programming: Principles and Practice	15.3 to 15.5 Memory, addresses and pointers; free store; destructors	452 to 463
Copying and the Rule of 3, 5 and 0	C++ Primer	13.1 Copy, assign and destroy	525 to 538
	C++ Primer	13.2 Copy control and resource management	539 to 544
	C++ Primer	13.5 Classes that manage dynamic memory	553 to 559
	C++ Primer	15.7 Constructors and copy control	651 to 658
	Programming: Principles and Practice	17.4 to 17.5 Copying and moving; essential operations	501 to 512

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
	Effective Modern C++	Item 17, Understand special member function generation	127 to 133
	C++ Move Semantics	3.1 and 3.1.1 to 3.1.2 Move semantics in ordinary classes	53 to 58
	C++ Move Semantics	3.2 Implementing the special copy and move member functions	58 to 66
	C++ Move Semantics	3.3 to 3.4 Rules for special member functions; the Rule of Five or Three	66 to 76
Inheritance	C++ Primer	15.1 to 15.2 OOP overview, defining base and derived classes	621 to 631
	C++ Primer	15.5 Access control and inheritance	640 to 645
	Programming: Principles and Practice	12.3, 12.5 Base and derived classes; benefits of object-oriented programming	380 to 387 and 389 to 390
	C++ Move Semantics	4.4 Move semantics in class hierarchies	93 to 96
Polymorphism and virtual functions	C++ Primer	15.3 Virtual functions	632 to 636
	C++ Primer	15.6 Class scope under inheritance	646 to 650
	C++ Primer	15.8 Containers and inheritance	659 to 662
	Effective Modern C++	Item 12, Declare overriding functions override	97 to 104
	C++ Move Semantics	4.4 Move semantics in class hierarchies	93 to 96
Abstract classes and interfaces	C++ Primer	15.4 Abstract base classes	637 to 639
explicit, conversion operators and copy-and-swap	C++ Primer	7.5 Constructors revisited	317 to 328
	C++ Primer	13.3 Swap	545 to 547
	C++ Primer	14.9 Overloading, conversions and operators	608 to 619
Operator overloading	C++ Primer	14.1 to 14.7 Basic concepts through member access	581 to 599
	Programming: Principles and Practice	8.4 to 8.6 Evolving a class (Date), enumerations, operator overloading	238 to 249